

2ª AVALIAÇÃO

Atividade prática em equipe

A atividade deverá ser realizada em equipes de **4 estudantes** e consistirá na implementação de uma API REST para gerenciamento simplificado de solicitações de viagem institucional.

A avaliação tem como objetivo verificar a aplicação prática dos seguintes conteúdos:

- modelagem de recursos REST;
- tratamento centralizado de erros;
- padronização de respostas;
- consumo de API externa;
- persistência em SGBD via Docker;
- testes automatizados com Vitest;
- organização mínima de um projeto backend em TypeScript.

1. Contexto do problema

Uma instituição pública de ensino superior realiza viagens acadêmicas e administrativas para diferentes finalidades, como participação em eventos, atividades de ensino, reuniões institucionais, ações de extensão, atividades de pesquisa e deslocamentos administrativos.

Embora um sistema completo de gestão de frota envolva aprovação, alocação de veículos, motoristas, controle de documentação, quilometragem e relatórios, esta avaliação trabalhará com um recorte simplificado do problema: o registro de solicitações de viagem institucional.

A API a ser desenvolvida deverá permitir que uma solicitação de viagem seja cadastrada, consultada e cancelada. No momento do cadastro, o sistema deverá validar regras básicas da solicitação e consultar uma API externa de feriados nacionais. Caso a data de saída da viagem corresponda a um feriado nacional, a solicitação não deverá ser registrada.

O objetivo da atividade é avaliar a capacidade da equipe de modelar recursos REST de forma adequada, padronizar respostas da API, tratar erros de maneira centralizada e consumir uma API externa simples, mantendo o código organizado e compreensível.

A atividade não exige implementação de autenticação, autorização, controle completo de frota, aprovação da solicitação, alocação de veículo, alocação de motorista ou relatórios administrativos.

2. Stack obrigatória

A atividade deverá ser implementada como uma API REST utilizando a seguinte stack:

- **Linguagem:** TypeScript em modo estrito (`strict: true`)
- **Ambiente de execução:** Node.js 20 ou superior
- **Framework HTTP:** de livre escolha
- **Banco de dados:** SGBD de livre escolha, obrigatoriamente executado via Docker Compose
- **Arquivo Docker obrigatório:** `docker-compose.yml`
- **Acesso ao banco:** livre escolha da equipe, podendo ser usado driver nativo, SQL direto, query builder, ORM, ODM ou biblioteca equivalente
- **Testes:** Vitest, podendo ser utilizada biblioteca auxiliar para testes de requisições HTTP
- **Consumo de API externa:** `fetch` nativo do Node.js ou biblioteca HTTP equivalente
- **API externa obrigatória:** BrasilAPI — Feriados Nacionais
- **Controle de versão:** Git
- **Hospedagem do código:** repositório público no GitHub

A aplicação deverá persistir os dados em um SGBD executado via Docker Compose. Não será aceita implementação apenas em memória.

O repositório deverá conter obrigatoriamente um arquivo chamado `docker-compose.yml`, responsável por definir o serviço de banco de dados utilizado pela aplicação.

A aplicação deverá utilizar obrigatoriamente as seguintes variáveis de ambiente:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=<valor funcional compatível com o docker-compose.yml>
HOLIDAYS_API_BASE_URL=https://brasilapi.com.br
```

Descrição das variáveis:

- `NODE_ENV` : ambiente de execução da aplicação.

- `PORT` : porta HTTP em que a API será executada.
- `DATABASE_URL` : string de conexão com o SGBD definido no `docker-compose.yml` .
- `HOLIDAYS_API_BASE_URL` : URL base da API externa de feriados nacionais.

O arquivo `.env.example` deverá conter valores funcionais para todas as variáveis de ambiente exigidas. O valor de `DATABASE_URL` deverá ser compatível com o SGBD escolhido pela equipe e com o serviço definido no `docker-compose.yml` .

Não deverão ser usados valores fictícios, placeholders ou exemplos não executáveis no `.env.example` entregue.

A API externa deverá ser usada para consultar feriados nacionais no fluxo de validação de uma solicitação de viagem.

Endpoint externo esperado:

```
GET /api/feriados/v1/{ano}
```

A equipe deverá disponibilizar um arquivo `.env.example` contendo as variáveis de ambiente especificadas acima.

A aplicação não deverá possuir frontend. A entrega será exclusivamente backend.

3. Entrega em repositório público do GitHub

A entrega deverá ser realizada por meio do **SIGAA**, no espaço destinado à submissão da atividade.

No SIGAA, a equipe deverá informar **unicamente o link para o repositório público no GitHub**. Não deverão ser anexados arquivos compactados, cópias do projeto, documentos adicionais ou qualquer outro material fora do repositório.

Cada equipe deverá entregar apenas **um repositório público**, contendo a implementação completa da API desenvolvida.

O repositório deverá conter obrigatoriamente:

- código-fonte completo da aplicação;
- arquivo `README.md` com instruções de instalação, configuração, inicialização do banco, execução e testes;
- identificação clara dos membros da equipe no `README.md` ;
- arquivo `.env.example` contendo as variáveis de ambiente exigidas;
- arquivo `docker-compose.yml` para execução do SGBD;
- arquivo `package.json` com os scripts obrigatórios da aplicação;
- estrutura necessária para criação e população inicial do banco de dados.

A identificação dos membros da equipe no `README.md` é obrigatória. O `README.md` deverá apresentar, de forma clara, o nome completo de todos os integrantes da equipe responsáveis pela entrega.

A equipe poderá utilizar `npm` , `pnpm` , `yarn` ou `bun` , desde que o gerenciador de pacotes escolhido esteja claramente informado no `README.md` .

O arquivo `README.md` deverá conter, no mínimo:

- nome da equipe, caso exista;
- nome completo de todos os integrantes da equipe;
- breve descrição da API implementada;
- tecnologias utilizadas;
- SGBD escolhido;
- gerenciador de pacotes adotado;
- instruções para instalação das dependências;
- instruções para configuração do ambiente;
- instruções para subir o SGBD com Docker Compose;
- instruções para inicialização e população do banco de dados;
- instruções para executar a aplicação;
- instruções para executar os testes;
- documentação resumida dos endpoints disponíveis, incluindo método HTTP, rota, descrição e exemplo de corpo da requisição quando aplicável.

O repositório não deverá incluir a pasta `node_modules` .

O arquivo `.env.example` deverá conter valores funcionais para todas as variáveis de ambiente exigidas. Após baixar o repositório, o avaliador deverá conseguir executar a aplicação sem precisar alterar manualmente os valores das variáveis.

O arquivo `.env.example` deverá conter as seguintes variáveis:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=<valor funcional compatível com o docker-compose.yml>
HOLIDAYS_API_BASE_URL=https://brasilapi.com.br
```

O valor de `DATABASE_URL` deverá ser funcional no projeto entregue, compatível com o SGBD escolhido e com o serviço definido no `docker-compose.yml`.

O arquivo `.env` não é obrigatório no repositório. A equipe deverá adotar uma das seguintes estratégias:

- versionar apenas o `.env.example` e fornecer instrução simples para gerar o `.env` a partir dele, sem edição manual dos valores;
- ou fazer a aplicação assumir os mesmos valores padrão do `.env.example` quando o arquivo `.env` não existir.

Em qualquer caso, o projeto deverá executar localmente com os valores definidos no `.env.example`, sem necessidade de adaptação por parte do avaliador.

A inicialização do banco de dados deverá ser realizada por meio do script obrigatório:

```
init:db
```

O script `init:db` deverá:

- criar as estruturas necessárias no banco de dados;
- popular o banco com pelo menos **10 registros** de solicitações de viagem;
- poder ser executado mais de uma vez sem quebrar a aplicação;
- não realizar pré-carga, seed ou espelhamento de feriados nacionais.

A população inicial deverá conter registros suficientes para permitir a consulta da lista de solicitações logo após a inicialização do banco.

Não será aceita entrega que dependa de criação manual do banco, execução manual de comandos fora dos scripts do projeto ou configuração manual de caminhos locais.

O `package.json` deverá conter, no mínimo, scripts equivalentes a:

```
{
  "scripts": {
    "dev": "...",
    "start": "...",
    "init:db": "...",
    "test": "..."
  }
}
```

A sequência mínima esperada para execução local deverá estar documentada no `README.md`, utilizando o gerenciador de pacotes escolhido pela equipe.

Exemplo genérico:

```
<package-manager> install
cp .env.example .env
docker compose up -d
<package-manager> run init:db
<package-manager> run dev
```

No exemplo acima, `<package-manager>` representa o gerenciador escolhido pela equipe, como `npm`, `pnpm`, `yarn` ou `bun`.

A avaliação será realizada com base no conteúdo disponível no repositório público informado no SIGAA até o prazo final de entrega.

4. Critérios de avaliação

A avaliação terá nota final de **0 a 10**, calculada por média ponderada conforme os critérios abaixo.

Em cada critério, a equipe receberá uma nota de **0 a 10**. A nota final será calculada aplicando-se o peso percentual de cada critério.

Critério	Peso
Correção funcional e regras do problema	30%
Testes automatizados	30%
Tratamento de erros e padronização de respostas	15%
Modelagem REST e contrato dos endpoints	10%
Consumo da API externa de feriados	10%
Histórico de commits	5%

4.1 Correção funcional e regras do problema — 30%

Será avaliado se a API implementa corretamente o fluxo principal da atividade, incluindo criação, listagem, consulta e cancelamento de solicitações de viagem.

Também serão avaliadas as regras obrigatórias: status inicial `pending`, cancelamento para `canceled`, validação de `returnAt` posterior ou igual a `departureAt`, validação de `passengerCount` maior que zero, impedimento de criação de solicitação com saída em feriado nacional e persistência dos dados no SGBD escolhido.

4.2 Testes automatizados — 30%

Será avaliada a existência e a qualidade dos testes automatizados com Vitest.

Os testes deverão cobrir, no mínimo, os principais cenários definidos no enunciado: criação válida, retorno anterior à saída, quantidade inválida de passageiros, bloqueio por feriado nacional, consulta de solicitação inexistente, cancelamento de solicitação existente e tentativa de cancelamento de solicitação já cancelada.

Também será avaliado se os testes podem ser executados por meio do script `test` definido no `package.json` e se não dependem da disponibilidade real da BrasilAPI.

4.3 Tratamento de erros e padronização de respostas — 15%

Será avaliado se a aplicação possui tratamento centralizado de erros e se todas as respostas seguem o padrão obrigatório definido no enunciado.

As respostas de erro deverão utilizar códigos internos padronizados, mensagens em inglês e códigos HTTP adequados, sem exposição de detalhes internos da aplicação.

4.4 Modelagem REST e contrato dos endpoints — 10%

Será avaliada a qualidade da modelagem dos recursos da API, considerando nomes de rotas, métodos HTTP, códigos de status, organização dos endpoints e aderência ao contrato mínimo definido no enunciado.

4.5 Consumo da API externa de feriados — 10%

Será avaliado se a aplicação utiliza dados da BrasilAPI — Feriados Nacionais, por consulta em tempo real ou por espelhamento sob demanda, respeitando a variável `HOLIDAYS_API_BASE_URL`.

Também será avaliado se a verificação de feriados está integrada ao fluxo de criação da solicitação e se falhas da API externa são tratadas adequadamente quando a aplicação precisar consultá-la.

Não serão aceitas validações baseadas apenas em dados estáticos, hardcoded ou previamente inseridos pelo script `init:db`.

4.6 Histórico de commits — 5%

Será avaliado se o histórico de commits demonstra evolução mínima do desenvolvimento da atividade.

Critérios observados:

- existência de mais de um commit relevante;
- mensagens de commit claras, objetivas e em inglês;
- commits relacionados a alterações compreensíveis no projeto;
- ausência de histórico concentrado apenas em um commit final;
- ausência de mensagens genéricas repetidas, como `update`, `fix`, `final` ou equivalentes sem contexto.

Não será exigida distribuição perfeitamente equilibrada de commits entre todos os integrantes, mas o histórico deverá indicar que houve desenvolvimento incremental do projeto.

4.7 Observações sobre correção

Caso o repositório esteja inacessível no momento da correção, a atividade não poderá ser avaliada.

Caso o projeto não possa ser instalado, inicializado ou executado localmente seguindo as instruções do `README.md`, a nota máxima da equipe será **4,0**, ainda que o código-fonte esteja parcialmente implementado.

Caso a aplicação não utilize um SGBD executado via Docker Compose, a equipe poderá perder pontos nos critérios de correção funcional, testes e execução local, conforme o impacto da ausência de persistência sobre o funcionamento da API.

Caso a aplicação não realize consulta real ou espelhamento sob demanda a partir da API externa de feriados no fluxo de criação da solicitação, a equipe perderá integralmente os pontos do critério de consumo da API externa.

5. Escopo funcional mínimo da API

A API deverá implementar um sistema simplificado de **solicitações de viagem institucional**.

O objetivo é permitir o cadastro, a consulta e o cancelamento de solicitações de viagem, com validação de regras básicas e consulta a uma API externa de feriados nacionais.

5.1 Recurso principal

O recurso principal da API será:

```
trip-requests
```

Esse recurso representa uma solicitação de viagem institucional.

5.2 Campos mínimos da solicitação de viagem

Cada solicitação de viagem deverá possuir, no mínimo, os seguintes campos:

- `id` : identificador único da solicitação;
- `requesterName` : nome do solicitante;
- `origin` : cidade de origem;
- `destination` : cidade de destino;
- `departureAt` : data e hora previstas de saída;
- `returnAt` : data e hora previstas de retorno;
- `purpose` : justificativa ou finalidade da viagem;
- `passengerCount` : quantidade de passageiros;
- `status` : situação atual da solicitação;
- `createdAt` : data e hora de criação do registro.

As datas deverão ser recebidas, armazenadas no banco de dados e retornadas pela API em formato **ISO 8601 completo**, como texto, utilizando o padrão UTC com sufixo `Z`.

Formato obrigatório:

```
YYYY-MM-DDTHH:mm:ss.sssZ
```

Exemplo:

```
2026-06-24T10:00:00.000Z
```

Assim, os campos `departureAt`, `returnAt` e `createdAt` deverão seguir esse formato.

Caso a equipe receba uma data válida em outro formato ISO 8601 aceito pelo JavaScript, a aplicação deverá normalizar o valor antes de persistir, utilizando o formato UTC com `Z`.

Exemplo:

```
2026-06-24T07:00:00-03:00
```

deverá ser armazenado como:

```
2026-06-24T10:00:00.000Z
```

Para fins de verificação de feriado nacional, deverá ser considerada a data civil extraída do campo `departureAt` já normalizado, no formato:

```
YYYY-MM-DD
```

5.3 Estados possíveis

A solicitação deverá trabalhar com os seguintes estados:

```
pending
canceled
```

Toda solicitação criada com sucesso deverá iniciar com status `pending`.

Ao ser cancelada, a solicitação deverá passar para o status `canceled`.

5.4 Endpoints obrigatórios

A API deverá disponibilizar, no mínimo, os seguintes endpoints:

```
POST /trip-requests
GET /trip-requests
GET /trip-requests/:id
PATCH /trip-requests/:id/cancel
GET /holidays/:year
```

Descrição dos endpoints:

- `POST /trip-requests` : cria uma nova solicitação de viagem;
- `GET /trip-requests` : lista as solicitações cadastradas;
- `GET /trip-requests/:id` : consulta uma solicitação específica;
- `PATCH /trip-requests/:id/cancel` : cancela uma solicitação existente;
- `GET /holidays/:year` : consulta os feriados nacionais de determinado ano por meio da BrasilAPI ou por espelhamento sob demanda a partir da BrasilAPI.

5.5 Regras obrigatórias

A API deverá implementar as seguintes regras:

- a data de retorno (`returnAt`) deverá ser posterior ou igual à data de saída (`departureAt`);
- a quantidade de passageiros (`passengerCount`) deverá ser maior que zero;
- a data de saída (`departureAt`) não poderá corresponder a um feriado nacional;
- a verificação de feriado deverá utilizar dados obtidos da BrasilAPI;
- uma solicitação inexistente deverá retornar erro padronizado;
- uma solicitação já cancelada não poderá ser cancelada novamente;
- quando a aplicação precisar consultar a API externa de feriados e ela estiver indisponível ou retornar erro, a solicitação não deverá ser criada.

5.6 Estratégia de consulta a feriados

A equipe poderá escolher uma das seguintes estratégias para consulta de feriados nacionais:

- **Consulta em tempo real:** a aplicação consulta a BrasilAPI diretamente sempre que precisar validar ou listar feriados.
- **Espelhamento sob demanda:** a aplicação consulta a BrasilAPI em tempo real quando precisar dos feriados de determinado ano e, em seguida, armazena localmente os dados obtidos para reutilização posterior.

No caso de espelhamento, os dados de feriados deverão ser obtidos da BrasilAPI durante a execução da aplicação, em tempo real, quando houver necessidade de consulta.

Não será permitido:

- inserir feriados manualmente no banco;
- manter lista fixa de feriados no código;
- popular feriados por meio do script `init:db`;
- validar feriados usando dados sem origem na BrasilAPI.

5.7 Fora do escopo

Não será exigido implementar:

- autenticação;
- autorização;
- cadastro de usuários;
- aprovação de solicitações;
- alocação de veículo;
- alocação de motorista;
- check-in ou check-out da viagem;
- controle de quilometragem;
- relatórios administrativos;
- frontend.

6. Padrão obrigatório de respostas e erros

Todas as respostas da API deverão seguir um formato padronizado.

6.1 Respostas de sucesso

As respostas de sucesso deverão utilizar o seguinte formato:

```
{
  "success": true,
  "data": {}
}
```

O campo `data` deverá conter o objeto ou a lista de objetos retornados pela operação.

Exemplo de resposta para criação de solicitação:

```
{
  "success": true,
  "data": {
    "id": "1",
    "requesterName": "Maria Silva",
    "origin": "Parnaíba",
    "destination": "Teresina",
    "departureAt": "2026-06-24T10:00:00.000Z",
    "returnAt": "2026-06-24T18:00:00.000Z",
    "purpose": "Participation in an institutional meeting",
    "passengerCount": 3,
    "status": "pending",
    "createdAt": "2026-06-20T14:30:00.000Z"
  }
}
```

6.2 Respostas de erro

As respostas de erro deverão utilizar o seguinte formato:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "A clear and objective error message"
  }
}
```

O campo `code` deverá conter um código interno padronizado, em inglês, escrito em `UPPER_SNAKE_CASE`.

O campo `message` deverá conter uma descrição clara e objetiva do erro, obrigatoriamente em inglês.

Exemplo:

```
{
  "success": false,
  "error": {
    "code": "HOLIDAY_TRIP_NOT_ALLOWED",
    "message": "Trip requests cannot start on a national holiday"
  }
}
```

6.3 Códigos de erro obrigatórios

A API deverá tratar, no mínimo, os seguintes códigos de erro:

Código	Situação
VALIDATION_ERROR	Dados obrigatórios ausentes, inválidos ou em formato incorreto
TRIP_REQUEST_NOT_FOUND	Solicitação de viagem não encontrada
TRIP_REQUEST_ALREADY_CANCELED	Tentativa de cancelar solicitação já cancelada
HOLIDAY_TRIP_NOT_ALLOWED	Data de saída corresponde a feriado nacional
HOLIDAYS_API_UNAVAILABLE	Falha ou indisponibilidade da API externa de feriados quando a aplicação precisar consultá-la
INTERNAL_SERVER_ERROR	Erro inesperado da aplicação

6.4 Códigos HTTP esperados

A API deverá utilizar códigos HTTP coerentes com o resultado da operação:

Situação	Código HTTP
Criação bem-sucedida	201 Created
Consulta bem-sucedida	200 OK
Cancelamento bem-sucedido	200 OK
Dados inválidos	400 Bad Request
Recurso não encontrado	404 Not Found
Violação de regra de negócio	409 Conflict
Falha na API externa	502 Bad Gateway
Erro inesperado	500 Internal Server Error

A resposta de erro não deverá expor stack trace, mensagens internas do banco de dados ou detalhes técnicos sensíveis.

7. Testes mínimos obrigatórios

A equipe deverá implementar testes automatizados utilizando **Vitest**.

Os testes deverão ser executados por meio do script `test` definido no `package.json`.

A equipe deverá implementar, no mínimo, testes para os seguintes cenários:

- criação de uma solicitação de viagem válida;
- tentativa de criação com `returnAt` anterior a `departureAt`;
- tentativa de criação com `passengerCount` menor ou igual a zero;
- tentativa de criação com `departureAt` em feriado nacional;
- consulta de solicitação inexistente;
- cancelamento de solicitação existente;
- tentativa de cancelamento de solicitação já cancelada.

Nos testes que envolvem feriados nacionais, a equipe poderá utilizar uma das seguintes estratégias:

- simular a resposta da BrasilAPI por meio de stub, mock ou fake;
- configurar a aplicação para consultar um serviço HTTP falso durante os testes;
- isolar o componente responsável pela consulta externa e substituí-lo por uma implementação controlada nos testes;
- utilizar dados espelhados durante o teste, desde que o espelhamento seja produzido pela própria execução do teste, e não pelo script `init:db`.

Os testes não deverão depender obrigatoriamente da disponibilidade real da BrasilAPI, pois falhas de rede ou indisponibilidade externa podem tornar a suíte instável.

A ausência de testes não impede a entrega da atividade, mas impactará a nota conforme os critérios de avaliação.

8. Contrato mínimo dos endpoints

A API deverá implementar os endpoints obrigatórios respeitando os contratos mínimos descritos nesta seção.

8.1 Criar solicitação de viagem

```
POST /trip-requests
```

Corpo mínimo da requisição:


```
{
  "requesterName": "Maria Silva",
  "origin": "Parnaíba",
  "destination": "Teresina",
  "departureAt": "2026-06-24T10:00:00.000Z",
  "returnAt": "2026-06-24T18:00:00.000Z",
  "purpose": "Participation in an institutional meeting",
  "passengerCount": 3
}
```

Comportamento esperado:

- validar os dados obrigatórios;
- validar o formato das datas;
- normalizar as datas para ISO 8601 em UTC;
- verificar se `departureAt` ocorre em feriado nacional, utilizando dados obtidos da BrasilAPI;
- criar a solicitação com status `pending`;
- persistir a solicitação no SGBD escolhido;
- retornar `201 Created` em caso de sucesso.

8.2 Listar solicitações de viagem

```
GET /trip-requests
```

Comportamento esperado:

- retornar todas as solicitações cadastradas;
- retornar uma lista vazia caso não existam solicitações;
- retornar `200 OK` em caso de sucesso.

8.3 Consultar solicitação por identificador

```
GET /trip-requests/:id
```

Comportamento esperado:

- retornar a solicitação correspondente ao identificador informado;
- retornar erro `TRIP_REQUEST_NOT_FOUND` caso a solicitação não exista;
- retornar `200 OK` em caso de sucesso.

8.4 Cancelar solicitação de viagem

```
PATCH /trip-requests/:id/cancel
```

Comportamento esperado:

- localizar a solicitação pelo identificador informado;
- retornar erro `TRIP_REQUEST_NOT_FOUND` caso a solicitação não exista;
- retornar erro `TRIP_REQUEST_ALREADY_CANCELED` caso a solicitação já esteja cancelada;
- alterar o status da solicitação para `canceled`;
- persistir a alteração no banco de dados;
- retornar `200 OK` em caso de sucesso.

8.5 Consultar feriados nacionais por ano

```
GET /holidays/:year
```

Comportamento esperado:

- retornar os feriados nacionais do ano informado;
- utilizar dados obtidos da BrasilAPI, por consulta em tempo real ou espelhamento sob demanda;
- retornar erro `HOLIDAYS_API_UNAVAILABLE` caso a aplicação precise consultar a API externa e ela esteja indisponível ou retorne erro;
- retornar `200 OK` em caso de sucesso.

Formato mínimo esperado para cada feriado retornado:

```
{
  "date": "2026-01-01",
  "name": "Confraternização Universal",
  "type": "national"
}
```

9. Orientações mínimas de implementação, nomenclatura e idioma

A equipe terá liberdade para organizar internamente o projeto, desde que a implementação seja clara, executável e coerente com os objetivos da avaliação.

A aplicação deverá apresentar separação mínima de responsabilidades entre:

- definição das rotas HTTP;
- execução das regras da solicitação de viagem;
- acesso ao SGBD escolhido;
- consumo ou espelhamento da API externa de feriados;
- tratamento de erros;
- padronização das respostas.

A chamada à BrasilAPI não deverá ficar espalhada diretamente pelos endpoints. Recomenda-se criar um componente específico para essa integração, por exemplo, um serviço ou cliente de feriados.

O acesso ao banco de dados também deverá ficar concentrado em componente próprio, evitando misturar comandos de persistência diretamente com a definição das rotas sempre que possível.

A equipe poderá adotar arquitetura simples, desde que o código permaneça legível, organizado e fácil de executar. Não será exigida arquitetura completa em camadas, Clean Architecture ou DDD.

A implementação deverá evitar complexidade desnecessária. O foco da atividade é demonstrar domínio sobre modelagem de recursos REST, padronização de respostas, tratamento de erros, persistência simples e consumo de API externa.

9.1 Recomendações gerais sobre nomenclatura e idioma

A equipe deverá observar as seguintes recomendações de nomenclatura e idioma:

- utilizar `camelCase` para variáveis, funções, métodos, propriedades, parâmetros e constantes locais;
- utilizar `PascalCase` para classes, interfaces, tipos e enums;
- utilizar `kebab-case` para arquivos e diretórios;
- escrever identificadores de código, comentários, testes, mensagens de commit, logs, erros e textos internos em inglês;
- manter em inglês os nomes relacionados ao código, incluindo variáveis, funções, métodos, classes, interfaces, tipos, enums, arquivos, diretórios, scripts, comandos, variáveis de ambiente, objetos de banco de dados, APIs, rotas, eventos, DTOs, schemas, use cases, repositories, services, configuration keys e quaisquer outros nomes técnicos;
- preservar exatamente os nomes técnicos definidos no enunciado, como `trip-requests`, `requesterName`, `departureAt`, `returnAt`, `passengerCount`, `DATABASE_URL` e `HOLIDAYS_API_BASE_URL`.

Os dados de negócio fornecidos pelo usuário ou retornados por serviços externos poderão permanecer no idioma original. Por exemplo, nomes de cidades, nomes de pessoas e nomes de feriados retornados pela BrasilAPI não precisam ser traduzidos.

10. Uso de IA, autoria e responsabilidade técnica

O uso de ferramentas de Inteligência Artificial será permitido como apoio ao desenvolvimento da atividade.

Entretanto, a equipe será integralmente responsável pelo código entregue, pela organização da solução, pela execução do projeto e pela correção funcional da API.

A utilização de IA não isenta a equipe de compreender o funcionamento da aplicação. Todos os integrantes deverão ser capazes de explicar as principais decisões técnicas adotadas, incluindo:

- modelagem dos recursos REST;
- regras de validação da solicitação de viagem;
- consumo ou espelhamento da API externa de feriados;
- tratamento de erros;
- padronização das respostas;
- persistência no SGBD escolhido;
- testes implementados.

Não será aceita como justificativa para erros da implementação a alegação de que determinado código foi gerado por ferramenta de IA.

A equipe deverá revisar criticamente qualquer código gerado com apoio de IA, garantindo que ele esteja coerente com o enunciado, com a stack obrigatória, com os padrões de nomenclatura definidos e com os critérios de avaliação.

11. Checklist final de entrega

Antes de submeter o link do repositório, a equipe deverá verificar se:

- o repositório está público no GitHub;
- o `README.md` contém o nome completo dos integrantes da equipe e instruções de execução;
- o `README.md` informa claramente qual gerenciador de pacotes deve ser utilizado;
- o `README.md` informa claramente qual SGBD foi escolhido;
- o arquivo `.env.example` existe e contém valores funcionais;
- o arquivo `docker-compose.yml` existe e sobe corretamente o SGBD;
- o projeto instala corretamente com o gerenciador de pacotes indicado pela equipe;
- o SGBD é executado via Docker Compose;
- o banco de dados é inicializado e populado por meio do script `init:db`;
- o script `init:db` insere pelo menos 10 registros de solicitações de viagem;
- o script `init:db` não realiza seed ou espelhamento de feriados;
- a aplicação executa por meio do script `dev` ou `start`;
- os testes executam por meio do script `test`;
- a pasta `node_modules` não foi versionada;
- os dados são persistidos no SGBD escolhido;
- a criação de solicitação valida feriados usando dados obtidos da BrasilAPI;
- as respostas de sucesso e erro seguem o padrão definido;
- os identificadores, arquivos, rotas e mensagens internas seguem as regras de nomenclatura e idioma;
- o histórico de commits demonstra evolução mínima do desenvolvimento.

A equipe poderá utilizar `npm`, `pnpm`, `yarn` ou `bun`, desde que o gerenciador escolhido esteja claramente documentado no `README.md`.

O `package.json` deverá conter, no mínimo, scripts equivalentes a:

```
{
  "scripts": {
    "dev": "...",
    "start": "...",
    "init:db": "...",
    "test": "..."
  }
}
```

A equipe deverá testar a execução do projeto em um clone limpo do repositório antes da entrega final, utilizando apenas os comandos documentados no `README.md`.